Agentic Systems

@aldrinjenson

LLMs

Flows we know

1. GPT-Wrapper

```
prompt = f"""
Write me a campaign plan for my product :${super_toothpaste}
"""

answer = llm.predict(prompt)
```

2. RAG

```
user_query = input("Enter your question")

similar_chunks = vector_db.retrieve(user_query, k=4)

prompt = f"""You are a question answering agent.
Here is some context:

{json.dumps(similar_chunk)}

Based on the above, answer the question: {user_query}"""

answer = llm.predict(prompt)
```

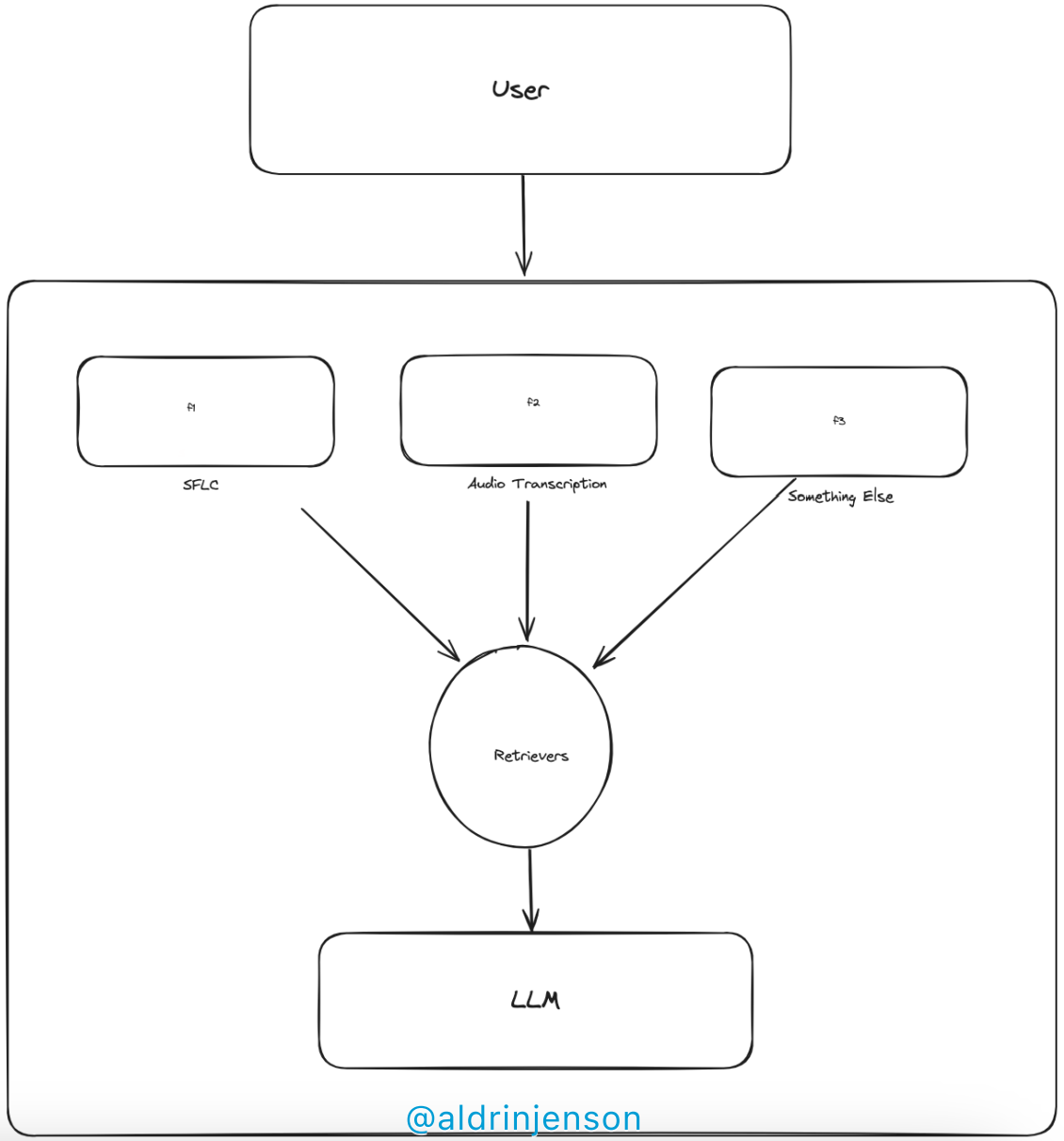
Limitations

1. You **always** have to perform Vector search

Trace latencies

Trace Name	50th	90th	95th ▼	99th
ragas_eval	32.686s	1m 38s	2m 4s	2m 23s
key-points	11.332s	30.020s	31.602s	37.537s
RAG	13.634s	24.297s	29.234s	1m 7s
vector_search	8.016s	25.525s	28.811s	1m 16s
LLM Query	1.750s	5.562s	7.105s	13.871s

2. Handling Multiple retrievers.



@aldrinjenson

3. Error prone : No validation

Query validation

At present, the SQL query generated by the LLM is directly returned to the user without validation, leaving a potential risk that the query may not run as expected. Implementing query validation, perhaps using a constrained beam search, could provide an extra layer of assurance.

https://medium.com/pinterest-engineering/how-we-built-text-to-sql-at-pinterest-30bad30dabff

4. Very limited functionality

“Escalating my chat to human”

“Cancel my order”

4. Quality of response is limited

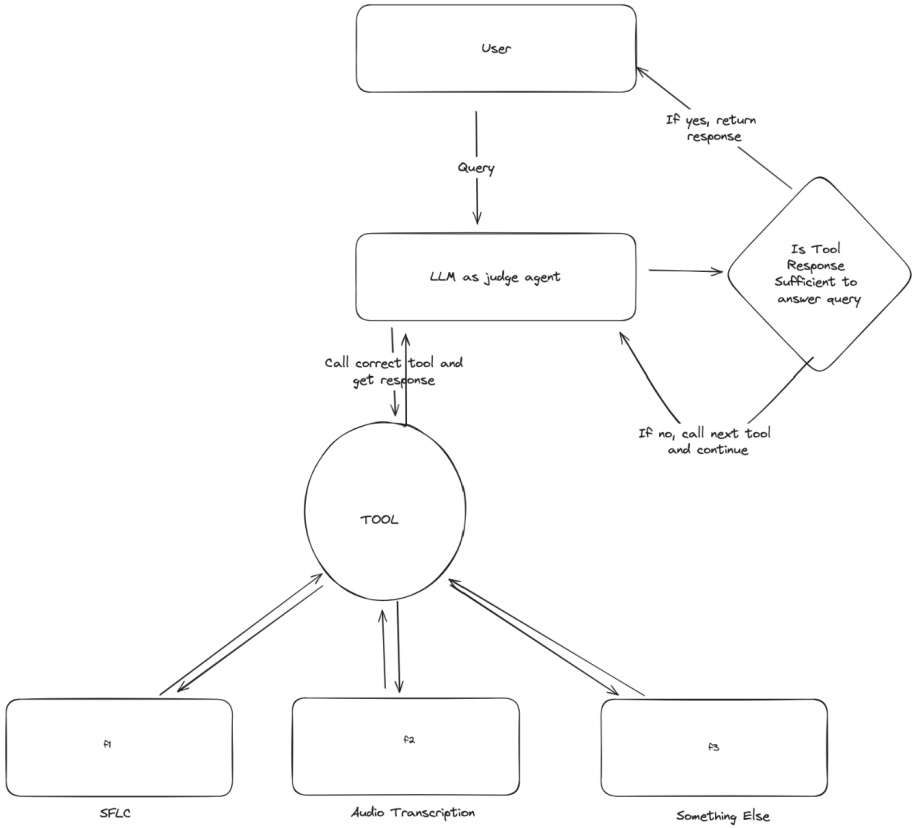
- Cannot solve Complex tasks
- Has Reasoning issues

(system 1 v/s system2 thinking)

Transformers architecture

Improving stuff

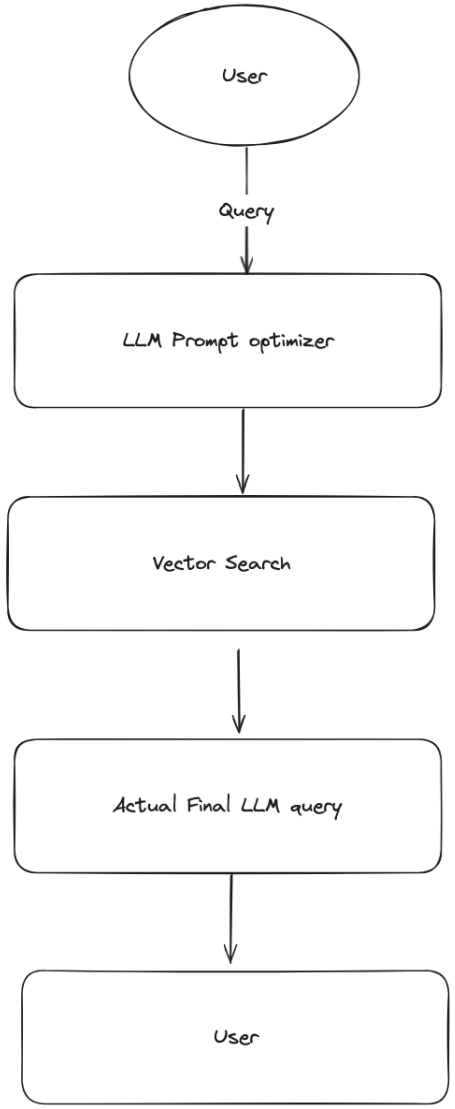
1. You **always** have to perform Vector search
2. Handling Multiple retrievers

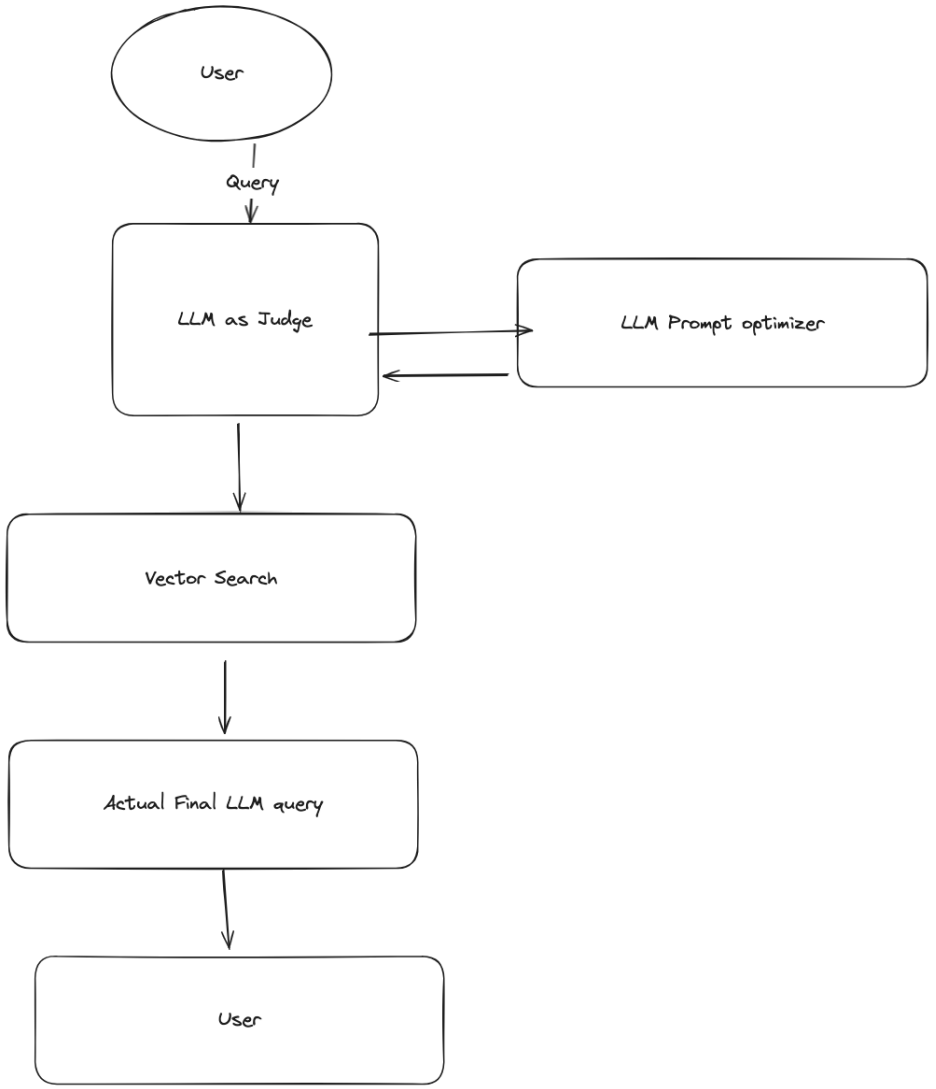


Still another problem: Vague user query inputs

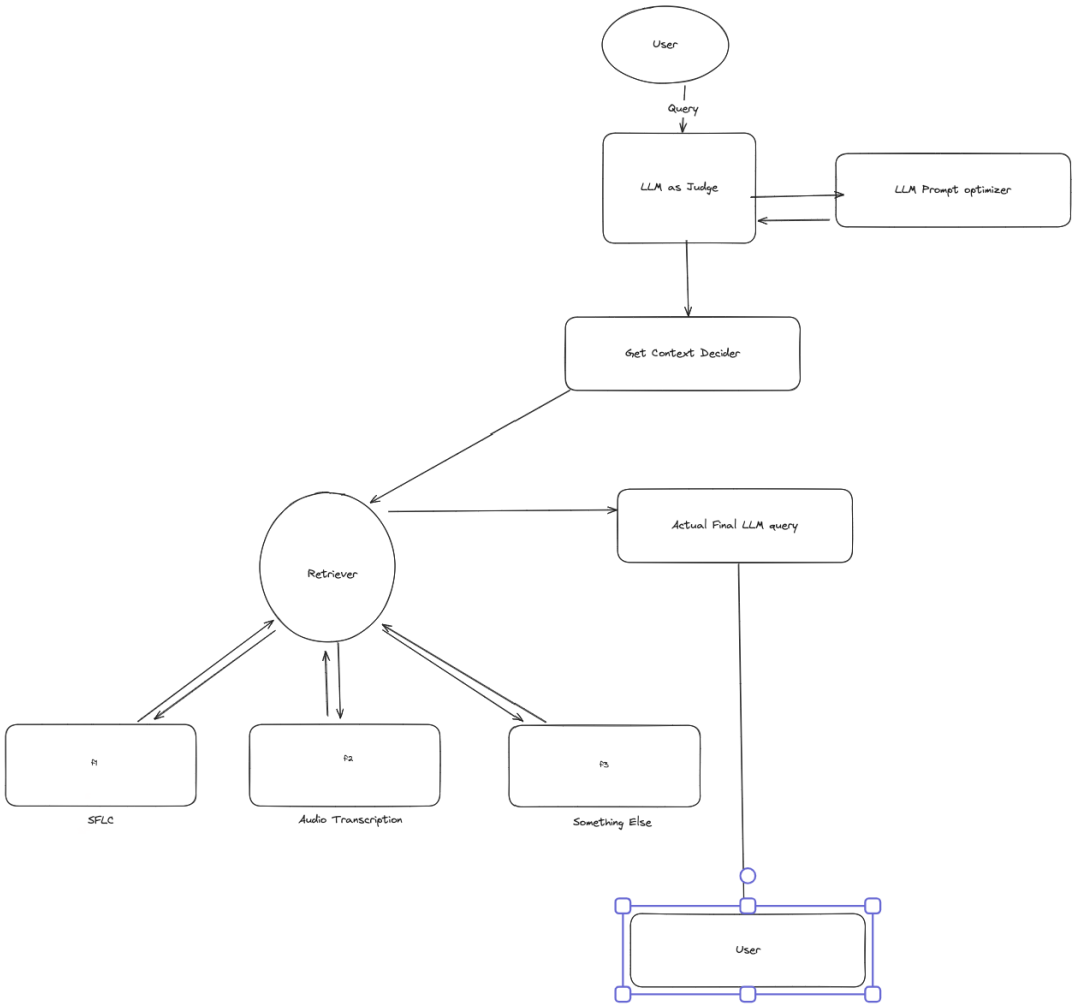
Example:

- **Less effective question:** "What are the latest treatments for lung cancer?"
- **More effective question:** "Identify emerging therapeutic targets in NSCLC based on recent genomic studies and their implications for future treatment strategies."


User
Query
LLM Prompt optimizer
Vector Search
Actual Final LLM query
User


To move canvas, hold mouse wheel or spacebar while dragging,
User
Query
LLM as Judge
LLM Prompt optimizer
Vector Search
Actual Final LLM query
User

Combine with multiple retrievers


User
Query
LLM as Judge
LLM Prompt optimizer
Get Context Decider
Retriever
Actual Final LLM query
P1
P2
P3
SFLC
Audio Transcription
Something Else
User

Now also add looping 🙂

Complexity ++

4. Error prone : No validation

```
answer = llm.predict(prmopt)

validation_prompt = f"Based on the question {question}, this was the answer given:
{answer}.
Is this corerct? if not fix it."

final_answer = llm.predict(validation_prompt)
```

```
answer = llm.predict(prmopt)

data, error = execute_sql(answer)

if error:
    validation_prompt = f"""Based on the question {question},
    this was the sql generated. {answer}.
    Got this error on running it: {error}.
    Write corrected sql.
    """

    final_answer = llm.predict(validation_prompt)

    execute_sql(final_answer)
```

5. Complex task / Reasoning issues

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei **Xuezhi Wang** **Dale Schuurmans** **Maarten Bosma**

Brian Ichter **Fei Xia** **Ed H. Chi** **Quoc V. Le** **Denny Zhou**

Google Research, Brain Team
{jasonwei,dennyzhou}@google.com

Abstract

We explore how generating a *chain of thought*—a series of intermediate reasoning steps—significantly improves the ability of large language models to perform complex reasoning. In particular, we show how such reasoning abilities emerge naturally in sufficiently large language models via a simple method called *chain-of-thought prompting*, where a few chain of thought demonstrations are provided as exemplars in prompting.

Experiments on three large language models show that chain-of-thought prompting improves performance on a range of arithmetic, commonsense, and symbolic reasoning tasks. The empirical gains can be striking. For instance, prompting a PaLM 540B with just eight chain-of-thought exemplars achieves state-of-the-art accuracy on the GSM8K benchmark of math word problems, surpassing even finetuned GPT-3 with a verifier.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Pause

Emergence

Reasoning + Acting = ReAct

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software) ', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]

```
agent.run("Who is Olivia Wilde's boyfriend? What is his current age raised to the 0.23 power?")
```

The chain execution looks as follows:

```
> Entering new AgentExecutor chain...
 I need to find out who Olivia Wilde's boyfriend is and then calculate his age raised to the 0.23 power.
Action: Search
Action Input: "Olivia Wilde boyfriend"
Observation: Olivia Wilde started dating Harry Styles after ending her years-long engagement to Jason Sudeikis — see their relationship timeline.
Thought: I need to find out Harry Styles' age.
Action: Search
Action Input: "Harry Styles age"
Observation: 29 years
Thought: I need to calculate 29 raised to the 0.23 power.
Action: Calculator
Action Input: 29^0.23
Observation: Answer: 2.169459462491557

Thought: I now know the final answer.
Final Answer: Harry Styles, Olivia Wilde's boyfriend, is 29 years old and his age raised to the 0.23 power is 2.169459462491557.

> Finished chain.
```

The output we get is as follows:

```
"Harry Styles, Olivia Wilde's boyfriend, is 29 years old and his age raised to the 0.23 power is 2.169459462491557."
```

```
%%capture
# update or install the necessary libraries
!pip install --upgrade openai
!pip install --upgrade langchain
!pip install --upgrade python-dotenv
!pip install google-search-results

# import libraries
import openai
import os
from langchain.llms import OpenAI
from langchain.agents import load_tools
from langchain.agents import initialize_agent
from dotenv import load_dotenv
load_dotenv()

# load API keys; you will need to obtain these if you haven't yet
os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY")
os.environ["SERPER_API_KEY"] = os.getenv("SERPER_API_KEY")
```

Now we can configure the LLM, the tools we will use, and the agent that allows us to leverage the ReAct framework together with the LLM and tools. Note that we are using a search API for searching external information and LLM as a math tool.

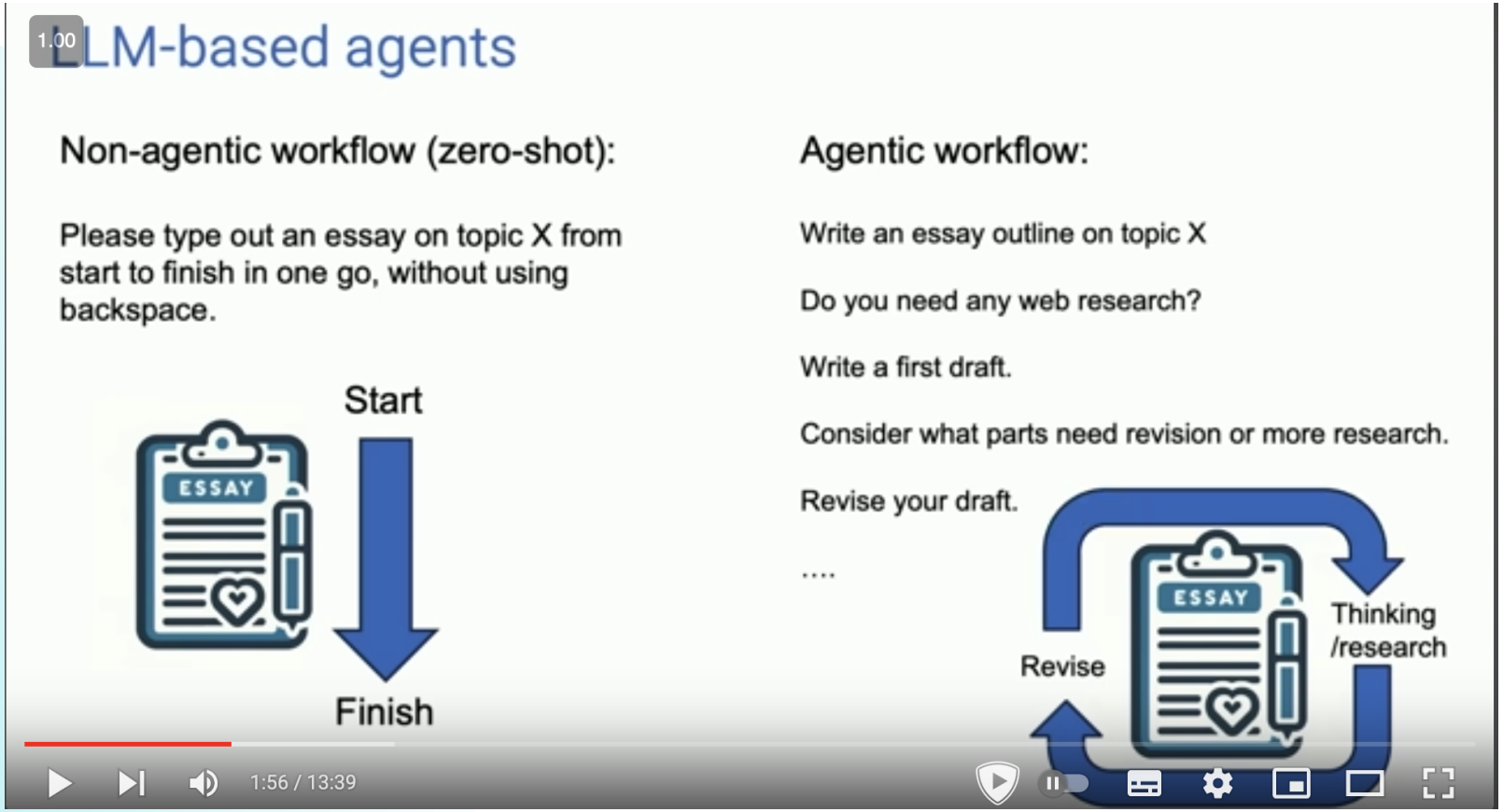
```
llm = OpenAI(model_name="text-davinci-003" ,temperature=0)
tools = load_tools(["google-serper", "llm-math"], llm=llm)
agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)
```

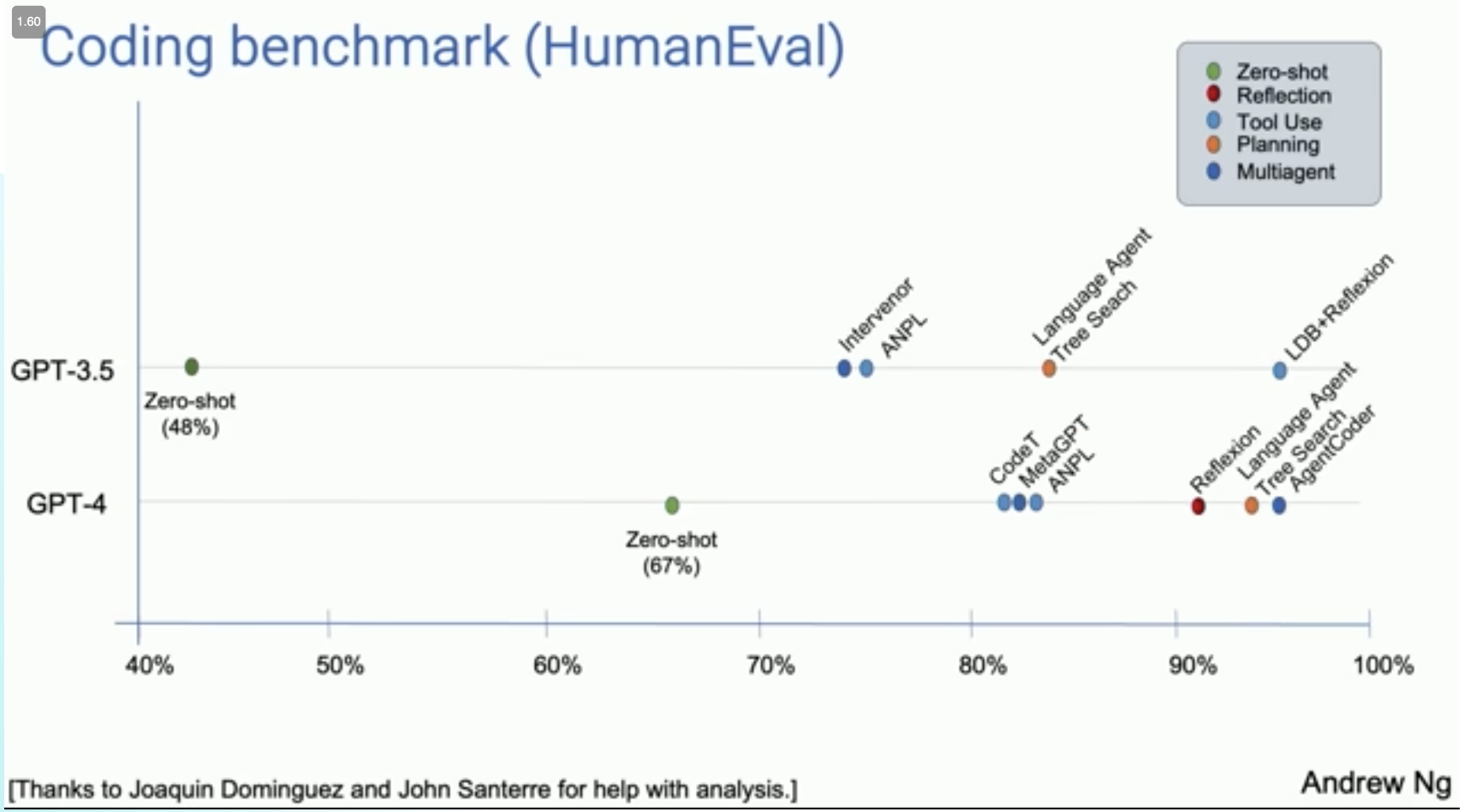
Agents

Why?

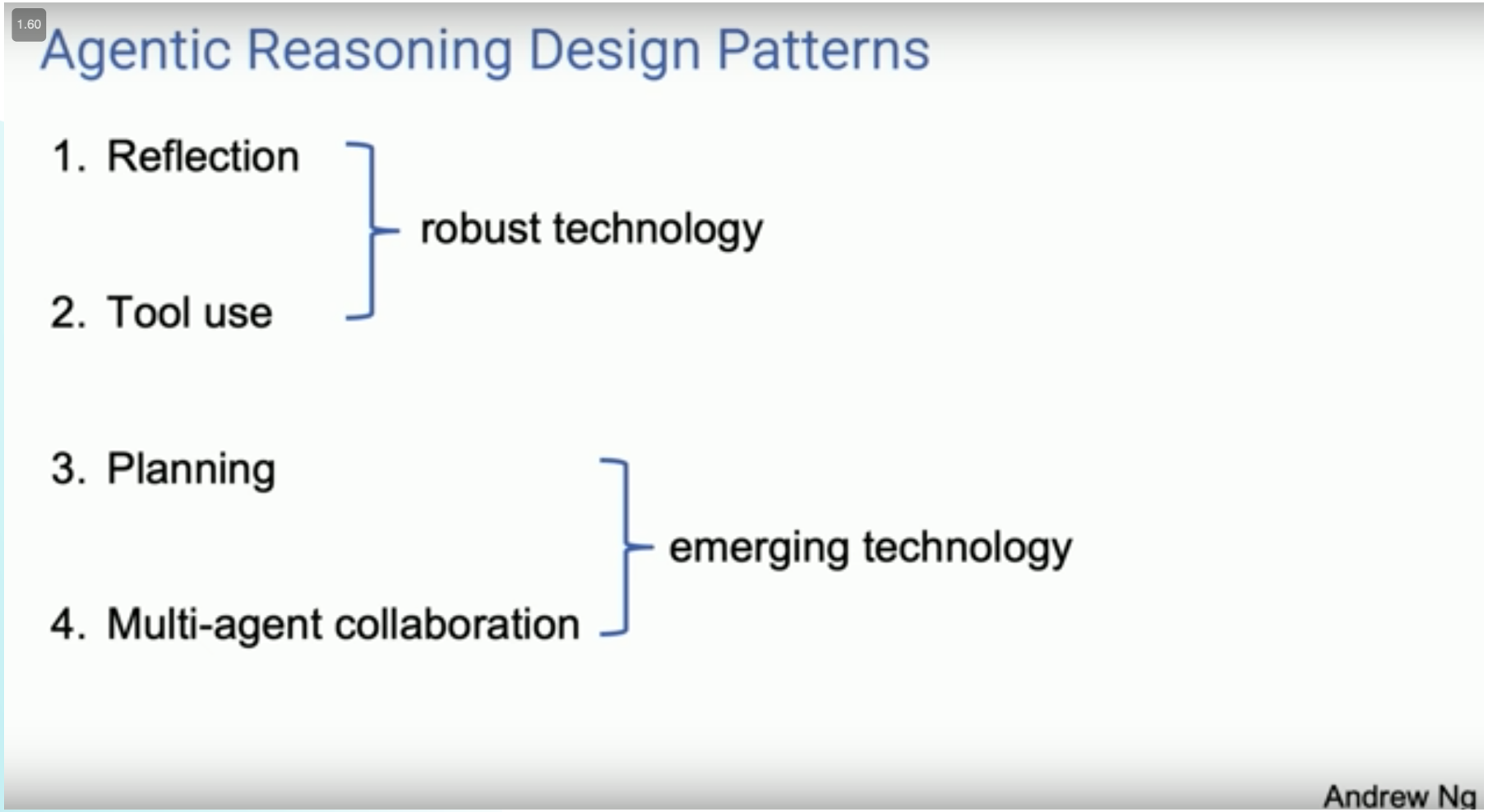
- Use LLMs as reasoning engines
- Non-deterministic sequence of actions

Improving Quality

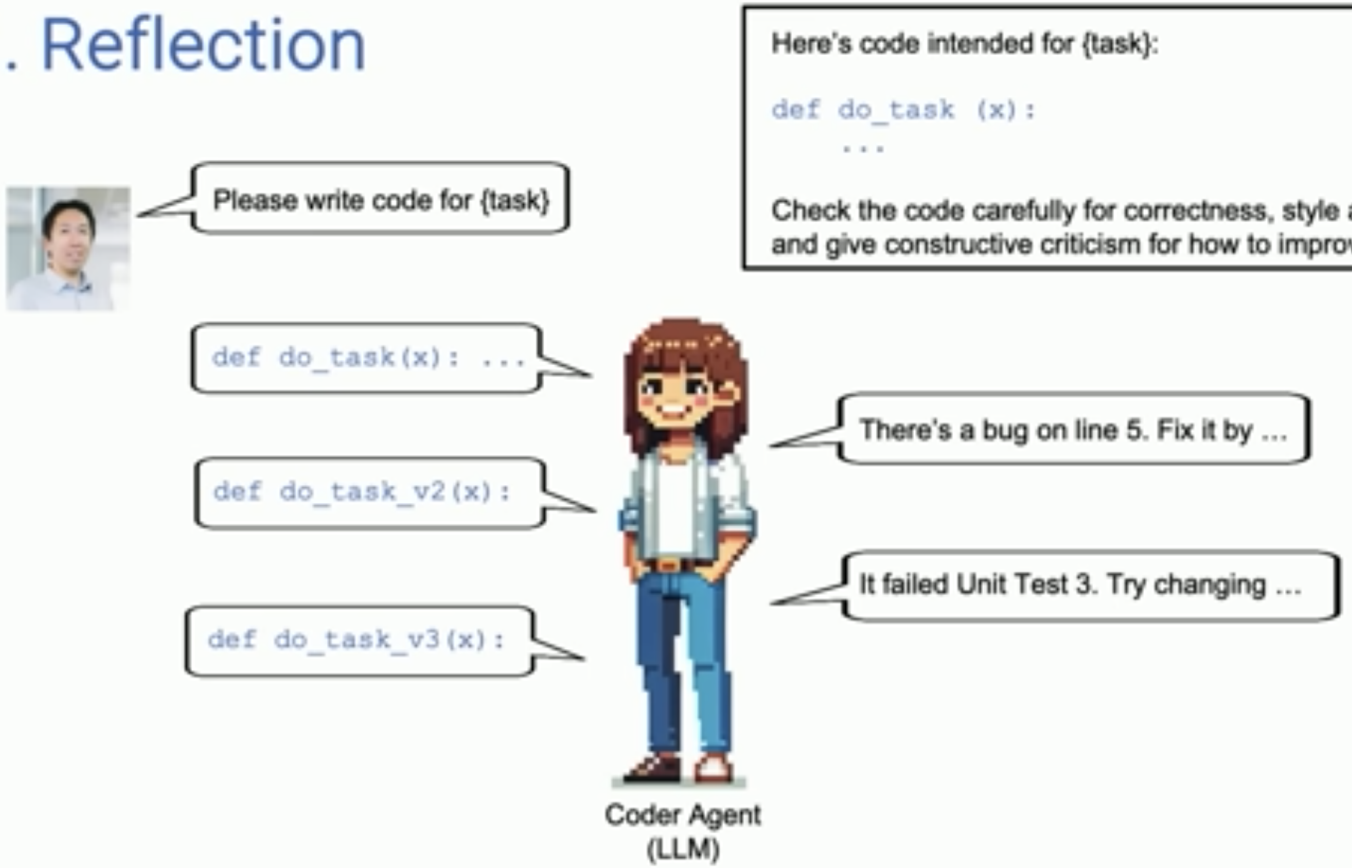




Gpt3.5 with agentic workflow outperforms Gpt4


1.60
Agentic Reasoning Design Patterns
1. Reflection
2. Tool use
robust technology
3. Planning
4. Multi-agent collaboration
emerging technology
Andrew Ng

1. Reflection



Recommended reading:

- Self-Refine: Iterative Refinement with Self-Feedback, Madaan et al. (2023)
- Reflexion: Language Agents with Verbal Reinforcement Learning, Shinn et al., (2023)

Andrew Ng

1.60

1. Reflection



Please write code for {task}

Here's code intended for {task}:

```
def do_task (x):
    ...
```

Check the code carefully for correctness, style and efficiency, and give constructive criticism for how to improve it.

```
def do_task(x): ...
```

```
def do_task_v2(x):
```

```
def do_task_v3(x):
```



Coder Agent
(LLM)

There's a bug on line 5. Fix it by …

It failed Unit Test 3. Try changing …



Critic Agent
(LLM)

Recommended reading:

- Self-Refine: Iterative Refinement with Self-Feedback, Madaan et al. (2023)
- Reflexion: Language Agents with Verbal Reinforcement Learning, Shinn et al., (2023)

Andrew Ng

Q: Duh, Can't I just resuse the same LLM with a different prompt?

4. Multiagent collaboration



Multiagent Debate

Task	Single agent	Multi-agent
Biographies	66.0%	**73.8%**
MMLU	63.9%	**71.1%**
Chess move	29.3%	**45.2%**

(Du et al., 2023)

Recommended reading:

- Communicative Agents for Software Development, Qian et al., (2023)
- AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation, Wu et al. (2023)

Andrew Ng

1.60

3. Planning

Request: Please generate an image where a girl is reading a book, and her pose is the same as the boy in the image example.jpg, then please describe the new image with your voice.



example.jpg

Pose Determination
openpose model

Pose-to-Image
google/vit model

Image-to-text
vit-gpt2 modle

Text-to-Speech
fastspeech model

[Example adapted from HuggingGPT paper]

Recommended reading:

- Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, Wei et al., (2022)
- HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face, Shen et al. (2023)

Andrew Ng

1.60

Agentic Reasoning Design Patterns

1. Reflection
   - Self-Refine: Iterative Refinement with Self-Feedback, Madaan et al. (2023)
   - Reflexion: Language Agents with Verbal Reinforcement Learning, Shinn et al., (2023)
2. Tool use
   - Gorilla: Large Language Model Connected with Massive APIs, Patil et al. (2023)
   - MM-REACT: Prompting ChatGPT for Multimodal Reasoning and Action, Yang et al. (2023)
3. Planning
   - Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, Wei et al., (2022)
   - HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face, Shen et al. (2023)
4. Multi-agent collaboration
   - Communicative Agents for Software Development, Qian et al., (2023)
   - AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation, Wu et al. (2023)

Andrew Ng

Some characterestics

- Iterative
- Tool Usage
- Reasoning
- Automated Action

Multi Agent Systems

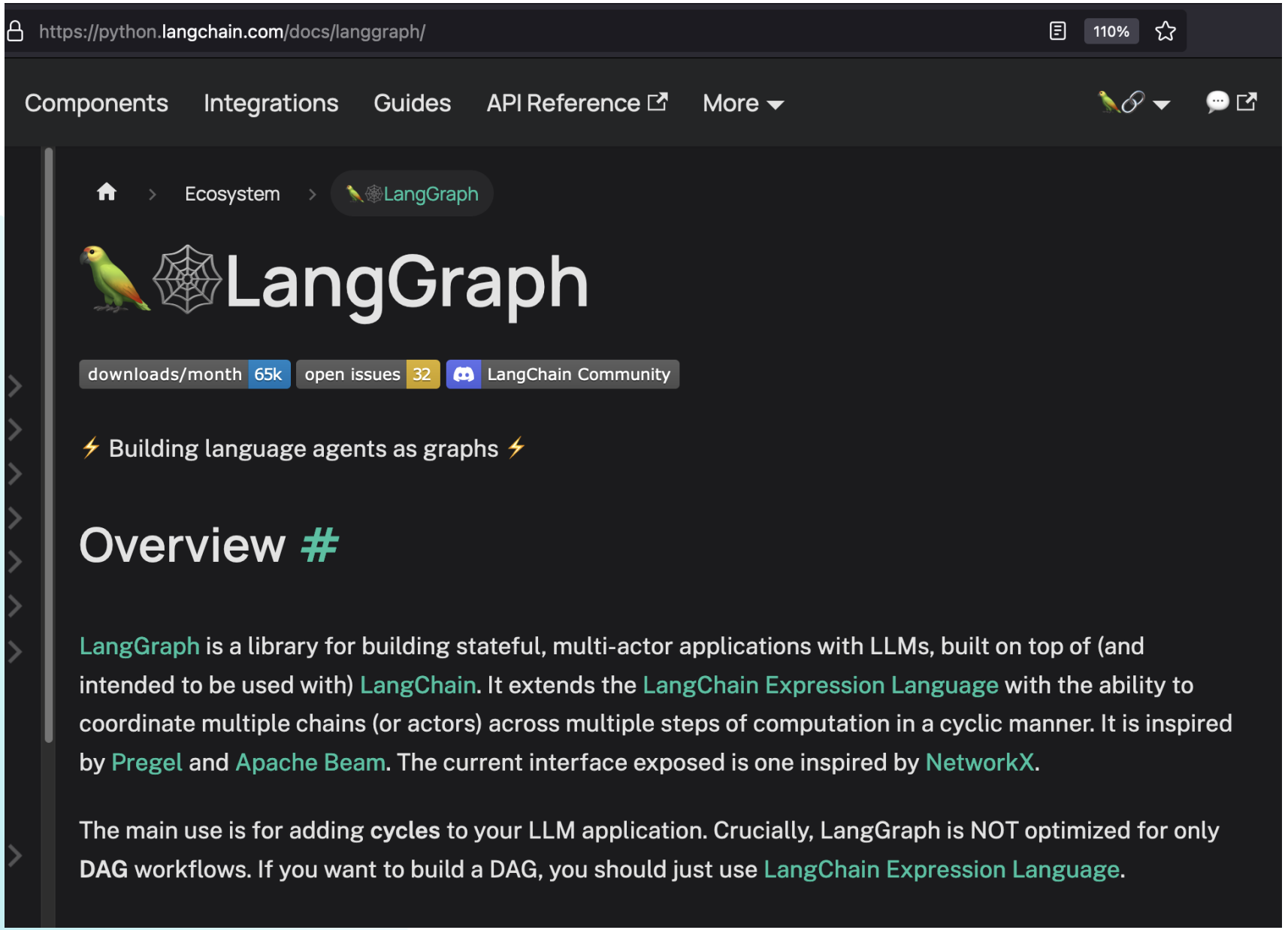
Image Optimizer Demo

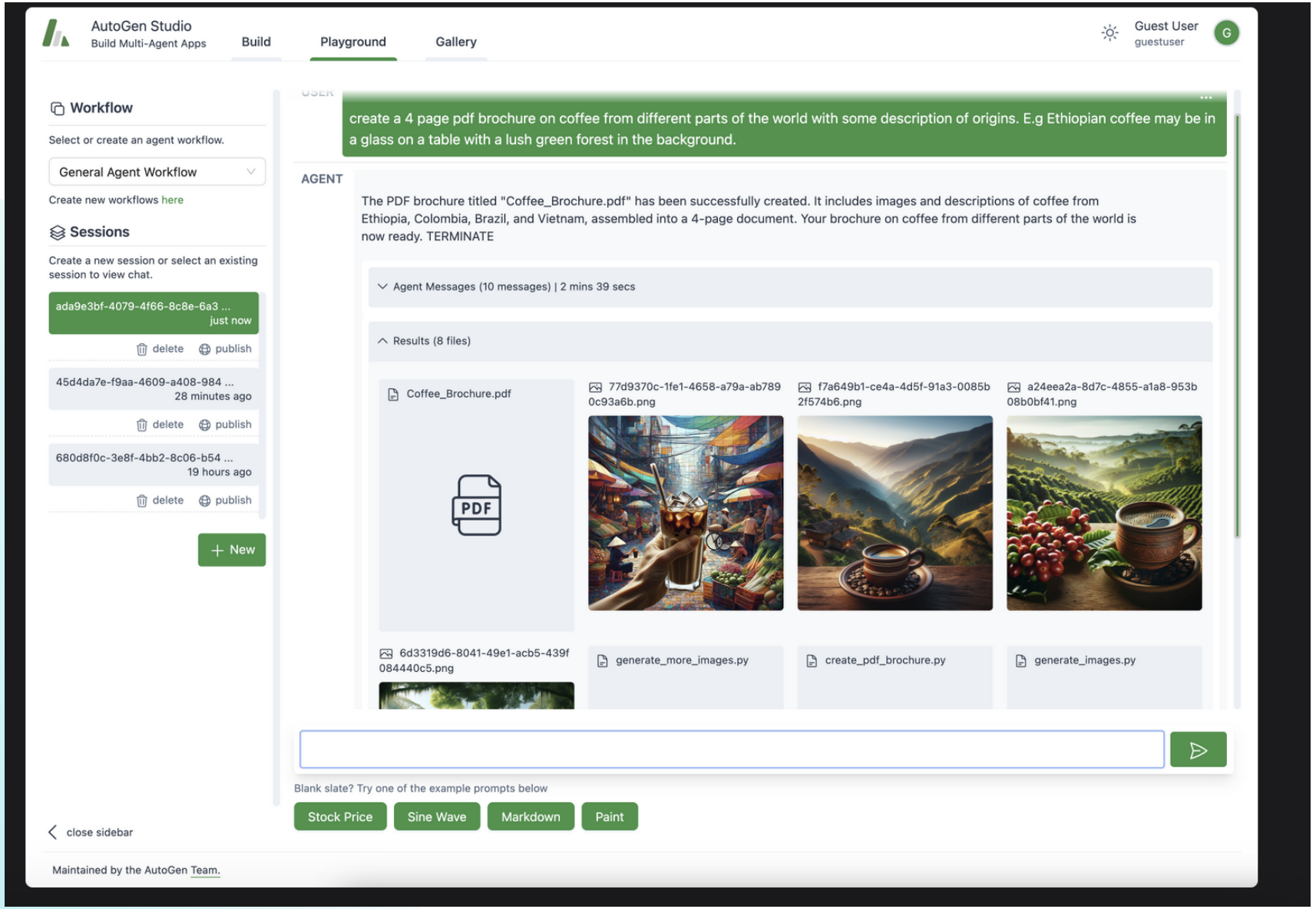
https://youtu.be/C94pTaKoLbU?si=5e8XntCK0gV43KYv&t=2040

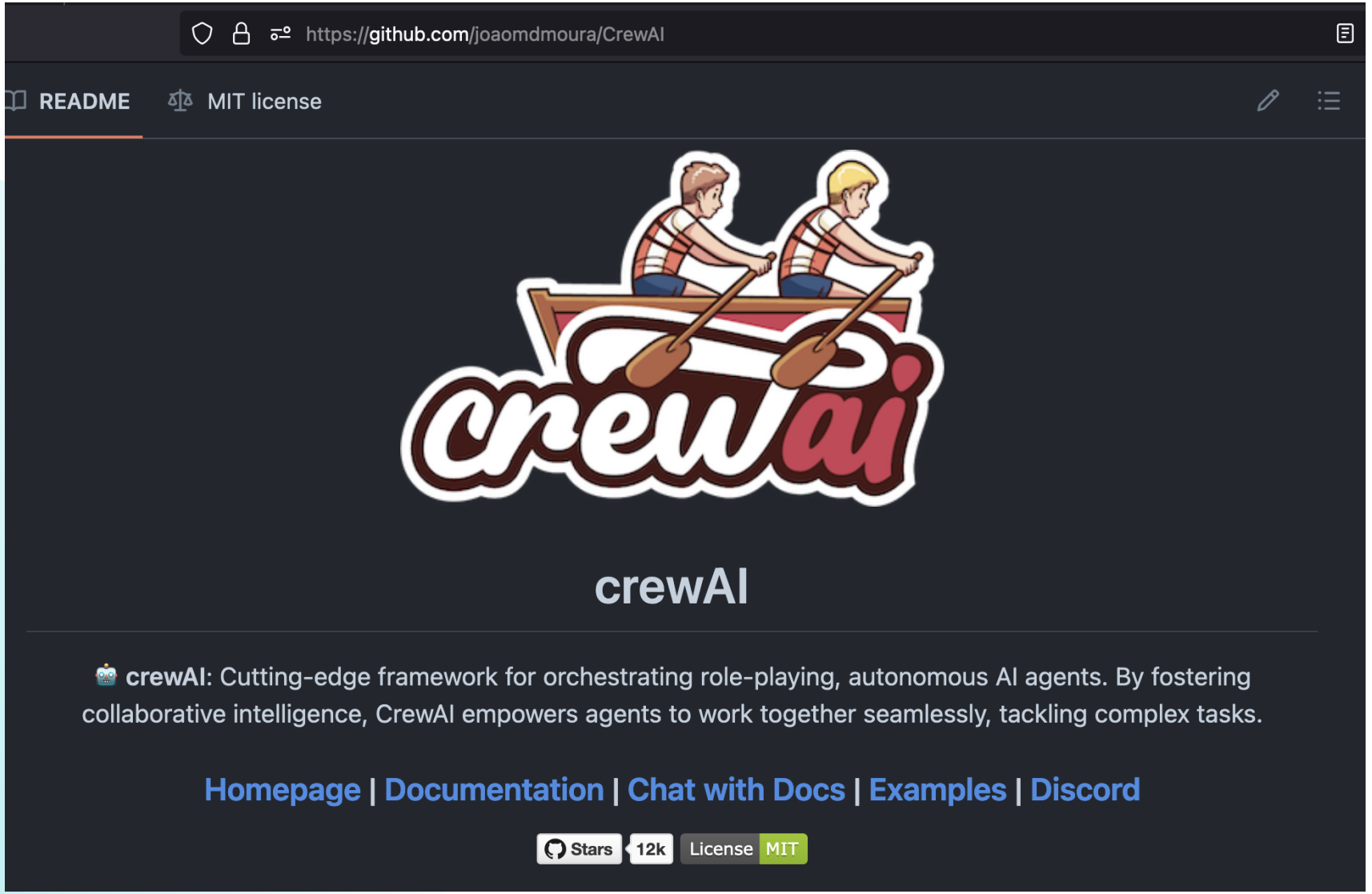
Main Potential usecases

- For Complex problems(Not necessariliy difficult ones)
- When the steps are not pre-determined

Popular frameworks


https://python.langchain.com/docs/langgraph/
110%
Components
Integrations
Guides
API Reference
More
Ecosystem
LangGraph
LangGraph
downloads/month 65k
open issues 32
LangChain Community
⚡ Building language agents as graphs ⚡
Overview #
LangGraph is a library for building stateful, multi-actor applications with LLMs, built on top of (and intended to be used with) LangChain. It extends the LangChain Expression Language with the ability to coordinate multiple chains (or actors) across multiple steps of computation in a cyclic manner. It is inspired by Pregel and Apache Beam. The current interface exposed is one inspired by NetworkX.
The main use is for adding cycles to your LLM application. Crucially, LangGraph is NOT optimized for only DAG workflows. If you want to build a DAG, you should just use LangChain Expression Language.


AutoGen Studio
Build Multi-Agent Apps
Build
Playground
Gallery
Guest User
guestuser
Workflow
Select or create an agent workflow.
General Agent Workflow
Create new workflows here
Sessions
Create a new session or select an existing session to view chat.
ada9e3bf-4079-4f66-8c8e-6a3 ...
just now
delete
publish
45d4da7e-f9aa-4609-a408-984 ...
28 minutes ago
delete
publish
680d8f0c-3e8f-4bb2-8c06-b54 ...
19 hours ago
delete
publish
New
close sidebar
create a 4 page pdf brochure on coffee from different parts of the world with some description of origins. E.g Ethiopian coffee may be in a glass on a table with a lush green forest in the background.
AGENT
The PDF brochure titled "Coffee_Brochure.pdf" has been successfully created. It includes images and descriptions of coffee from Ethiopia, Colombia, Brazil, and Vietnam, assembled into a 4-page document. Your brochure on coffee from different parts of the world is now ready. TERMINATE
Agent Messages (10 messages) | 2 mins 39 secs
Results (8 files)
Coffee_Brochure.pdf
PDF
77d9370c-1fe1-4658-a79a-ab7890c93a6b.png
f7a649b1-ce4a-4d5f-91a3-0085b2f574b6.png
a24eea2a-8d7c-4855-a1a8-953b08b0bf41.png
6d3319d6-8041-49e1-acb5-439f084440c5.png
generate_more_images.py
create_pdf_brochure.py
generate_images.py
Blank slate? Try one of the example prompts below
Stock Price
Sine Wave
Markdown
Paint
Maintained by the AutoGen Team.


https://github.com/joaomdmoura/CrewAI
README
MIT license
crewAI
🤖 crewAI: Cutting-edge framework for orchestrating role-playing, autonomous AI agents. By fostering collaborative intelligence, CrewAI empowers agents to work together seamlessly, tackling complex tasks.
Homepage | Documentation | Chat with Docs | Examples | Discord
Stars 12k License MIT

Other Multi Agent AI Frameworks

- [Recommended] AutoGen: framework that enables the development of LLM applications using multiple agents that can converse with each other to solve tasks. Disclaimer: I am a contributor!
- CAMEL: Communicative Agents for "Mind" Exploration of Large Scale Language Model Society
- ChatDev: Explores the idea of agents collaborating towards writing software.
- Langroid - Harness LLMs with Multi-Agent Programming.

Note that there are a few other OSS projects that focus on agents but have less emphasis on multiple agents collaborating towards a task. Some examples include:

- LangChain Agents - A subset of Langchain where a language model is used as a reasoning engine to determine which sequence of actions to take and in which order.
- LLAMAIndex Agents - An "agent" is an automated reasoning and decision engine. It takes in a user input/query and can make internal decisions for executing that query in order to return the correct result.
- BabyAGI - A script for AI-powered task management system.
- AutoGPT - A semi-autonomous agent powered by LLMs to execute any task for you

Conclusion

Referneces

- https://youtu.be/sal78ACtGTc?si=5KGmg8KXc9nqLaT7
- https://www.youtube.com/watch?v=DWUdGhRrv2c
- https://www.promptingguide.ai/techniques/react
- https://arxiv.org/abs/2210.03629
- https://arxiv.org/pdf/2201.11903.pdf

Thank You!

@aldrinjenson

aldrinjenson.com

This presentation was built using Free Software!

> **So … what are agents?**
> Definitions vary, but based on the above, we can define agents as **entities that can reason, act** and **communicate** to **solve problems.**
>
> **Multi-agent** applications involve a group of agents, each ideally possessing diverse capabilities, such as language models, tools, and objectives, working collaboratively to solve tasks.

Along the way, it is also expected that these agents exhibit some level of **autonomy** and be efficient with the use of human feedback (i.e., they must be proactive, learn human preferences and context). In fact, their level of "helpfulness" is correlated with how well they can correctly address tasks with little or no human intervention, truly freeing us from the burden of many tasks.